

Familia 1 — Credenciales y secretos expuestos

Documento profundo de referencia · Para uso del formador Sesión "Seguridad y buenas prácticas en desarrollo con IA" · AI PM Pro · The Hero Camp

Índice

1. [Por qué esta familia abre la sesión](#)
 2. [Definiciones precisas: no todas las claves son iguales](#)
 3. [Mecanismo: por qué ocurre, especialmente con agentes IA](#)
 4. [Magnitud: los datos que hay que conocer de memoria](#)
 5. [Velocidad de explotación: el factor tiempo](#)
 6. [Severidad: tres ejes para clasificar](#)
 7. [A quién afecta: afectados directos e indirectos](#)
 8. [Casos reales documentados](#)
 9. [El factor agente IA](#)
 10. [Lo que el PM senior tiene que interiorizar](#)
 11. [Fuentes](#)
-

1. Por qué esta familia abre la sesión

Las credenciales expuestas son el vector de seguridad con la mejor relación "impacto real / esfuerzo del atacante" que existe hoy. No requiere vulnerabilidades zero-day, ni ingeniería social sofisticada, ni acceso privilegiado previo. Requiere una cosa: que alguien deje una clave donde no debe, y que un bot la encuentre. Ese proceso completo, como veremos, puede durar menos de cinco minutos.

Para un Product Manager Senior que construye con Claude Code, esta familia es la puerta de entrada a todo lo demás por tres razones concretas. Primero, porque es el fallo más documentado, con cifras y casos públicos que no admiten discusión. Segundo, porque es el fallo que más directamente afecta al bolsillo del PM o de su empresa: las facturas llegan en horas, no en meses. Y tercero, porque es el fallo donde el agente IA cambia más las reglas del juego, y hay datos concretos que lo demuestran.

Arrancar la sesión con esta familia engancha a la audiencia con cifras que duelen, antes de entrar en categorías más técnicas como RLS o datos sensibles, que requieren más contexto para ser comprendidas.

2. Definiciones precisas: no todas las claves son iguales

Una de las razones por las que los PMs subestiman esta familia es porque todas las claves les parecen iguales. No lo son. Conviene manejar estas cinco categorías con precisión:

API keys de servicios externos. Claves que tu aplicación usa para autenticarse contra servicios de terceros: OpenAI, Anthropic, Google (Gemini, Maps, YouTube), Stripe, SendGrid. Si se exponen, el atacante puede consumir esos servicios **a cargo de tu cuenta**. Aquí entran los casos de facturas de decenas de miles de dólares.

Credenciales de infraestructura en la nube. Claves de AWS, GCP, Azure que permiten provisionar recursos. Si se exponen, el atacante puede levantar servidores, almacenamiento, GPUs, bases de datos, todo a tu nombre. Son las más destructivas económicamente porque no tienen techo natural: un bot puede lanzar cientos de instancias GPU en minutos.

Credenciales de base de datos. Cadenas de conexión a PostgreSQL, MongoDB, Redis, Supabase, etc. Si se exponen, el atacante tiene acceso directo a los datos de tus usuarios. Aquí el impacto es más regulatorio que económico.

Tokens de autenticación y sesión. JWT, OAuth tokens, personal access tokens de GitHub. Permiten actuar como si fueras un usuario legítimo del sistema. Su exposición facilita ataques de suplantación de identidad.

Secretos de aplicación. Claves simétricas para firmar cookies, cifrar datos en reposo, generar tokens internos. Su exposición compromete la integridad del sistema aunque no haya un "servicio externo" al que llamar.

La distinción más importante, y la que los PMs casi nunca manejan, es entre **claves de identificación** (pensadas para ser públicas, como una Google Maps API key embebida en frontend) y **claves de autenticación** (que dan acceso real y nunca deben salir del servidor). El problema es que esa distinción se está rompiendo: como veremos en el caso Gemini 2026, claves que Google consideraba "solo identificadores públicos" empezaron a autenticar contra Gemini silenciosamente, convirtiéndolas en credenciales de facto.

3. Mecanismo: por qué ocurre, especialmente con agentes IA

La cadena de eventos típica cuando un agente IA deja una credencial expuesta tiene siempre la misma forma. La describo aquí con detalle porque entender el mecanismo es lo que permite prevenirlo.

Paso 1: El PM pide una funcionalidad. Normalmente algo del tipo "conecta el formulario a Supabase para guardar los leads" o "añade un botón que genere una descripción con Gemini".

Paso 2: El agente intenta la solución más directa. El agente está optimizado para que el código funcione. Cuando tiene que elegir entre dos caminos, tiende al que produce menos errores. Si usar una clave anónima requiere configurar políticas (RLS, scopes, permisos), y usar una clave administrativa "simplemente funciona", el agente tiende a la segunda.

Paso 3: La clave se hardcodea "temporalmente". El agente escribe la clave directamente en el código porque es la forma más rápida de hacer que la demo funcione. En teoría, el código se refactorizará después para mover la clave a variables de entorno. En la práctica, el PM ve que la funcionalidad va, la aprueba, y el refactor no ocurre nunca.

Paso 4: El PM hace git commit y push. Si el repositorio es público, la clave acaba de ser expuesta al mundo. Si es privado, "solo" queda expuesta a todos los colaboradores, al historial de Git permanente, y a cualquier fuga futura del repositorio. GitGuardian reporta que las claves en repositorios privados son **nueve veces más comunes** que en públicos, precisamente porque los desarrolladores asumen que la privacidad del repo es protección suficiente, y bajan la guardia.

Paso 5: Bots automatizados detectan la clave. En repositorios públicos, bots especializados están monitorizando el stream de eventos de GitHub en tiempo real. La velocidad de detección es comentada en detalle en la sección 5.

Paso 6: El atacante explota la clave. Dependiendo del tipo, el ataque puede ser: consumir servicios de pago a tu cargo (Gemini, OpenAI), minar criptomonedas usando tu infraestructura (AWS, GCP), exfiltrar datos de tu base de datos, o pivotar hacia otros sistemas.

El factor agente IA amplifica varios pasos de esta cadena. El agente acelera el Paso 2 (va más rápido al camino que "funciona"), multiplica el volumen del Paso 3 (produce más código, por tanto más oportunidades de hardcodear), y según los datos de GitGuardian 2026, **duplica literalmente la tasa de fuga por commit:** los commits co-firmados con Claude Code tuvieron una tasa de exposición de secretos del 3,2% frente al 1,5% de commits sin asistencia de IA.

4. Magnitud: los datos que hay que conocer de memoria

Estos son los números que tienes que poder citar en clase sin dudar. Todos son verificables en las fuentes listadas al final.

Secretos expuestos en GitHub público:

- **29 millones de nuevos secretos hardcodeados** detectados en GitHub público solo en 2025. Es un incremento del 34% interanual y **el mayor salto anual jamás registrado** por GitGuardian desde que publica el informe.
- En 2024 fueron 23,8 millones (25% de incremento frente a 2023). La tendencia no se está estabilizando; se está acelerando.
- Desde 2021, los secretos expuestos han crecido un 152%, mientras que la población de desarrolladores de GitHub solo ha crecido un 98%. **Los secretos se están fugando más rápido que crecen los desarrolladores.**

Persistencia del problema:

- **El 70% de los secretos expuestos en 2022 seguían activos en 2025.** No es que se solucionen tarde; es que mayoritariamente no se solucionan nunca. Una vez expuesta, la credencial tiende a seguir siendo válida hasta que alguien la explota.
- Un secreto comprometido aparece de media en **ocho ubicaciones distintas** en la misma máquina (archivos .env, historial de shell, configs de IDE, tokens cacheados, artefactos de build).

Repositorios privados, el falso refugio:

- **El 35% de los repositorios privados** contienen secretos en texto plano.
- Las claves IAM de AWS aparecen en el 8% de repos privados, **cinco veces más frecuente que en repos públicos.** El falso sentido de seguridad es real y medible.
- Las credenciales de MongoDB son el secreto más comúnmente expuesto en repos públicos (18,8% del total).

El factor IA, en cifras:

- **Los commits co-firmados por Claude Code en 2025 tuvieron una tasa de fuga de secretos del 3,2%,** frente al 1,5% de línea base del resto de GitHub. Literalmente el doble.
- **1.275.105 secretos de servicios de IA** (OpenAI, Anthropic, Google AI, etc.) detectados solo en 2025. Un **81% más** que en 2024.

- **24.008 secretos únicos detectados en archivos de configuración de servidores MCP** en GitHub público. De ellos, 2.117 verificados como válidos. El 20% eran claves de Google API; el 14% cadenas de conexión a PostgreSQL.

Docker e infraestructura:

- En un escaneo de 15 millones de imágenes Docker públicas, GitGuardian encontró **100.000 secretos válidos**, incluyendo claves de AWS y tokens de GitHub de empresas Fortune 500.
- El 18% de las imágenes Docker escaneadas contenían secretos; el 15% de esos eran válidos.

Fuera del código:

- **El 28% de los incidentes de fugas** en 2025 se originaron fuera del código fuente: en Slack, Jira, Confluence, Notion, y herramientas similares de colaboración.
- **El 56,7% de los secretos encontrados solo en herramientas de colaboración** se clasificaron como críticos, frente al 43,7% de los encontrados en código. Las fugas en Slack son, en media, más graves que las fugas en repositorios.

Un dato que conviene tener a mano para el epic fail: en el caso concreto de ChatGPT, **más de 5.000 repositorios públicos de GitHub y 3.000 sitios web en producción** tenían claves de OpenAI hardcodeadas expuestas activamente, según Cyble Research.

5. Velocidad de explotación: el factor tiempo

Este es el dato que más impacta a la audiencia cuando se presenta bien. No hay margen para reaccionar; hay margen para prevenir.

Tiempo medio desde exposición hasta primer intento de explotación: entre 4 y 5 minutos. Palo Alto Networks Unit 42, en la campaña EleKtra-Leak, documentó atacantes que comprometían credenciales de AWS en menos de cinco minutos tras su exposición en GitHub público. En el caso concreto de los \$89K de AWS, los bots encontraron la clave **cuatro minutos** después del commit.

Cómo lo hacen los atacantes: los bots monitorizan continuamente el stream de eventos públicos de GitHub Archive, que registra en tiempo real cada push a repositorios públicos. Patrones específicos como **AKIA** (AWS), **ghp_** (GitHub PAT), **sk-** (OpenAI) o **AIza** (Google) son buscados automáticamente. Cuando se encuentra una clave, herramientas como TruffleHog la verifican contra la API del servicio para confirmar que está viva, y otras herramientas automatizadas la explotan.

Persistencia del atacante tras compromiso: una vez una credencial cae en manos automatizadas, los atacantes suelen también escanear el historial completo de Git, no solo el commit actual. Claves "borradas" en commits posteriores pero presentes en el histórico siguen siendo extraíbles. Incluso claves eliminadas con **git rm** permanecen en el append-only log de Git indefinidamente.

Ventana temporal comparada:

- Tiempo de reacción humana típica en una empresa: horas a días.
- Tiempo de reacción de un bot: minutos.
- Tiempo medio para que AWS aplique su política de cuarentena automática: minutos, pero no siempre llega antes que el atacante.

La conclusión operativa es contundente: no puedes depender de detectar la fuga y reaccionar. Tienes que prevenir que la fuga ocurra. Una vez la clave ha sido commiteada a un repositorio público, asume que está comprometida, rota inmediatamente.

6. Severidad: tres ejes para clasificar

No todas las exposiciones de credenciales son iguales en impacto. Para evaluarlas correctamente conviene cruzar tres ejes, porque una brecha puede ser baja en uno y crítica en otro.

Eje 1 · Impacto económico directo

El dinero que la organización puede perder por el propio incidente, sin contar multas ni reputación.

Crítico (decenas a cientos de miles): claves de infraestructura cloud con permisos amplios (AWS IAM con AdministratorAccess, GCP service accounts con Owner). El atacante puede lanzar cientos de instancias GPU para minar criptomoneda. Casos documentados: \$89.432 en AWS (3 días), \$82.314 en Gemini (48 horas), \$55.444 en Gemini (3 meses).

Alto (miles a decenas de miles): claves de APIs de IA generativa (OpenAI, Anthropic, Gemini) sin límites de gasto configurados. Un atacante puede consumir miles de dólares al día en llamadas.

Medio (cientos a miles): APIs con facturación por uso pero con techos naturales (SendGrid, Stripe en modo test, Twilio). El daño económico directo es limitado pero no nulo.

Bajo: claves de servicios gratuitos o con cuota muy restringida. Aquí el problema es más reputacional que económico.

Eje 2 · Impacto regulatorio y legal

Las obligaciones derivadas de normativas (RGPD en Europa, HIPAA en sanidad, PCI-DSS en pagos).

Crítico: si la credencial da acceso a datos personales o sanitarios. Brechas de datos personales bajo RGPD implican multas de hasta el 4% de la facturación anual global o 20 millones de euros, lo que sea mayor. En 2025, solo en Europa, se impusieron más de 330 sanciones importantes por incumplimientos del RGPD relacionados con falta de protección técnica en bases de datos.

Alto: si da acceso a datos sensibles pero no estrictamente personales (datos empresariales confidenciales, propiedad intelectual).

Medio: si la exposición permite acceso a sistemas donde podría haber datos personales aunque no sea la ruta principal.

Bajo: exposiciones de credenciales de servicios que no tocan datos personales en absoluto.

Un apunte importante: **el RGPD obliga a notificar brechas en 72 horas** desde que se tiene constancia. Si una credencial expuesta ha permitido acceso a datos personales, el PM y su empresa tienen un reloj corriendo desde el momento en que se detecta.

Eje 3 · Impacto reputacional y de negocio

Las consecuencias más difíciles de cuantificar pero las que más duelen a medio plazo.

Crítico: incidentes que llegan a prensa especializada o general (el caso de la startup mexicana fue recogido por The Register, TechSpot, Cybernews, Bleeping Computer). Pérdida de confianza de clientes, abandono de usuarios, impacto en rondas de financiación.

Alto: incidentes que circulan en comunidades profesionales (Reddit, Hacker News, Twitter técnico). Daño a la marca empleadora, dificultad para contratar talento técnico.

Medio: incidentes conocidos solo dentro de la organización o por un cliente específico. Pérdida de confianza de ese cliente, posible ruptura de contrato.

Bajo: incidentes detectados y corregidos internamente sin externalización.

El coste medio global de una brecha en 2025, según IBM, fue de **4,44 millones de dólares**. En Estados Unidos subió a un récord de **10,22 millones**. Y el coste medio de brechas donde las credenciales comprometidas fueron el vector inicial fue de **4,67 millones de dólares**, con un tiempo medio de identificación y contención de **246 días**. Casi ocho meses desde la exposición hasta la resolución.

7. A quién afecta: afectados directos e indirectos

Esta dimensión es la que más se olvida y la que más pesa cuando hay una crisis real. Un PM Senior debe pensar en círculos concéntricos.

Afectados directos:

- **La organización que construyó el producto.** Factura, multa, reparación, coste de auditoría.
- **El PM responsable.** En empresas pequeñas, responsabilidad personal; en grandes, impacto en carrera y confianza interna.
- **El equipo de desarrollo.** Sobrecarga durante la respuesta, sesiones post-mortem, cambios de procedimiento impuestos.

Afectados indirectos, a menudo olvidados:

- **Los usuarios cuyos datos se exponen.** Este es el grupo con más peso regulatorio. Si se expuso una base de datos con emails, nombres, historial de uso, el impacto es directamente sobre ellos aunque no lo sepan.
- **Los clientes B2B de la organización.** Si tu producto B2B fue comprometido, tus clientes podrían tener obligaciones de notificación derivadas (especialmente en sectores regulados como banca o sanidad).
- **Terceros con los que hay integraciones.** Si tu producto se conecta a los sistemas de un partner, una credencial comprometida puede ser el vector de entrada a ellos.
- **Los usuarios finales indirectos.** En servicios B2B2C, los usuarios de los clientes de tus clientes. La cadena de responsabilidad puede llegar muy lejos.

Un dato concreto para ilustrar: en 2025, según IBM, **los ataques a través de cadena de suministro** (supply chain compromise) supusieron el 30% de los incidentes relacionados con modelos y aplicaciones de IA. Una credencial que expone un proveedor puede convertirse en la puerta de entrada a sus clientes.

8. Casos reales documentados

Aquí están los casos que puedes usar en clase, con datos verificados y referencias. Los ordeno por relevancia para la sesión.

Caso 1 · Startup mexicana, \$82.314 en 48 horas con Gemini (febrero 2026)

Este es el epic fail principal recomendado. Reciente, profesional, y con un giro narrativo perfecto.

Equipo de tres desarrolladores en México, gasto mensual habitual en servicios IA de unos 180 dólares. Entre el 11 y el 12 de febrero de 2026 se generaron 82.314,44 dólares en cargos no autorizados, un incremento del 45.000% sobre su consumo normal. El gasto fue casi exclusivamente en Gemini 3 Pro Image y Gemini 3 Pro Text. Al contactar con Google, la respuesta fue el "Shared Responsibility Model": Google asegura su plataforma, los usuarios aseguran sus claves. No hubo perdón ni reducción de la factura.

El giro narrativo es lo que hace este caso excepcional: los desarrolladores sostienen que no expusieron la clave de forma obvia. La investigación paralela de Truffle Security reveló que Google había cambiado silenciosamente el comportamiento de sus API keys: claves que durante más de una década se habían considerado meros identificadores de facturación (como las de Google Maps, públicas por diseño), de repente autenticaban también contra Gemini sin que Google avisara a nadie. Truffle Security encontró 2.863 claves de Google activas con este problema en noviembre de 2025.

Mensaje que este caso permite transmitir en clase: cuando construyes con servicios cloud, las reglas de ayer no siempre son las de hoy. Una clave "segura" porque "es solo un identificador público" puede convertirse en credencial sensible de un día para otro sin tu intervención.

Fuentes:

- The Register: https://www.theregister.com/2026/03/03/gemini_api_key_82314_dollar_charge/
- TechSpot: <https://www.techspot.com/news/111529-stolen-gemini-api-key-turned-180-bill-82000.html>
- Truffle Security (investigación): <https://trufflesecurity.com/blog/google-api-keys-werent-secrets-but-then-gemini-changed-the-rules>
- CSO Online: <https://www.csoonline.com/article/4138749/silent-google-api-key-change-exposed-gemini-ai-data.html>

Caso 2 · Estudiante de Georgia, \$55.444 en Gemini (junio–septiembre 2025)

Ideal como segundo caso para ilustrar el factor humano y el arco temporal largo.

Estudiante de Georgia (el país caucásico, con ingresos medios diarios de unos 15 dólares) que subió una API key de Gemini a GitHub el 6 de junio de 2025, pensando que el repositorio era privado. Durante el verano no revisó el correo de la universidad. Un usuario de GitHub le avisó el 7 de septiembre. Para entonces la factura era de 55.444 dólares, acumulada en tres oleadas. En dos días, los atacantes hicieron 14.200 requests a la API: todas fallidas, y aun así se las cobraron. Google finalmente perdonó la deuda tras la presión comunitaria, pero la experiencia estuvo cerca de arruinarle la vida al estudiante, cuyo ingreso medio diario era de 15 dólares.

Mensaje que este caso permite transmitir: la exposición puede ser silenciosa durante meses. No es solo "me pasó esta tarde"; puede estar ocurriendo ahora mismo sin que lo sepas.

Fuentes:

- CSO Online: <https://www.csoonline.com/article/4138749/silent-google-api-key-change-exposed-gemini-ai-data.html>

- 36Kr (reporte detallado): <https://eu.36kr.com/en/p/3486014581496960>

Caso 3 · AWS \$89.432 por minería de criptomoneda (2025)

Perfecto como caso ilustrativo del mecanismo completo desde commit hasta explotación.

Desarrollador junior commitea a GitHub público código de prueba con credenciales IAM de AWS embebidas a las 14:00. A las 14:04 (cuatro minutos después), bots automatizados encuentran las credenciales. Las credenciales correspondían a un IAM user con permiso AdministratorAccess. En las horas siguientes, los atacantes levantaron progresivamente instancias EC2 GPU en la región us-west-2. A medianoche había 487 instancias corriendo. La minería continuó durante tres días antes de saltar la alerta de facturación (configurada en 10.000 dólares). Factura final: 89.432,18 dólares.

El artículo original del incidente hace el análisis completo de fallos sistémicos: sin pre-commit hooks, IAM user con permisos excesivos, sin CloudWatch alarms en tiempo real, billing alert demasiado alto, sin límites de tasa en IAM.

Mensaje que este caso permite transmitir: el error individual (un commit despistado) es solo la chispa. Lo que hace que la factura llegue a 89K es la ausencia de capas defensivas. Una sola capa bien implementada (pre-commit hook, principio de mínimo privilegio, alerta en tiempo real) habría limitado el daño.

Fuente:

- Medium (relato de primera persona): <https://medium.com/lets-code-future/we-got-a-89k-aws-bill-overnight-heres-what-went-wrong-b5dfd51c9a7e>

Caso 4 · Campaña EleKtra-Leak (2023, Palo Alto Unit 42)

Útil como respaldo técnico cuando quieras hablar de la industrialización del ataque.

Campaña documentada por investigadores de Palo Alto Networks Unit 42. Atacantes que comprometían credenciales AWS expuestas en GitHub público en menos de cinco minutos desde la exposición, incluso a pesar de la política de cuarentena automática de AWS. Crearon al menos 474 instancias EC2 compute-optimized distintas para minar Monero en solo 37 días (agosto a octubre de 2023). Usaron máquinas del tipo c5a.24xlarge precisamente por su mayor potencia de cómputo.

Mensaje: no es un atacante aislado buscando oportunidades; es una industria completa con infraestructura dedicada.

Fuentes:

- The Register: https://www.theregister.com/2023/10/30/cryptojackers_steal_aws_credentials_github/
- The Hacker News: <https://thehackernews.com/2023/10/elektra-leak-cryptojacking-attacks.html>
- Dark Reading: <https://www.darkreading.com/cloud-security/elektra-leak-attackers-harvest-aws-cloud-keys-github-campaign>

Caso 5 · ChatGPT API keys expuestas masivamente (2025)

Útil para contexto de escala del problema específico con IA.

Cyble Research descubrió **más de 5.000 repositorios públicos de GitHub y 3.000 sitios web en producción** que tenían claves de OpenAI hardcodedas expuestas activamente. La investigación señaló que el fenómeno del "vibe coding" (priorizar velocidad sobre seguridad al construir con asistentes IA) estaba detrás del auge: desarrolladores probando integraciones con ChatGPT hardcodedan la clave "temporalmente", y se quedan así.

Mensaje: no es un problema de una empresa o un estudiante. Es un problema sistémico en la cultura actual de construcción con IA.

Fuente:

- Vicarius / Cyble Research: <https://www.vicarius.io/articles/8-000-chatgpt-api-keys-exposed-across-github-production-sites>

9. El factor agente IA

Esta sección es la que más diferencia tu sesión de una formación de seguridad clásica. Merece ser tratada con cuidado, sin ser alarmista pero sin minimizar.

Lo que los datos dicen con claridad: los commits asistidos por Claude Code en 2025 tuvieron una tasa de fuga de secretos del 3,2%, aproximadamente el doble del 1,5% de línea base del resto de GitHub. Dwayne McDaniel (GitGuardian) lo formuló bien en una entrevista reciente: "No estamos diciendo que la IA en sí sea mala. Lo que tenemos son los datos. Y los datos dicen que si co-firmabas tus commits con Claude Code durante 2025, tenías más del doble de probabilidades de commitear un secreto". Este dato debe usarse con honestidad: no es que Claude Code (o cualquier otro asistente) sea el culpable; es que la velocidad que habilitan cambia el régimen de fugas.

Los mecanismos por los que el agente amplifica el riesgo:

1. **Velocidad de producción de código.** Más código producido significa más oportunidades de hardcodedar un secreto. Es aritmética, no malicia.
2. **Optimización por "que funcione".** Cuando el agente se encuentra con un error de permisos, tiende a resolver con la credencial que da más acceso. No es un defecto del modelo; es su función objetivo.
3. **Democratización de la construcción.** Personas sin formación formal de seguridad están construyendo productos en producción. No saben lo que no saben. El agente no suple la formación en buenas prácticas que el desarrollador no tiene.
4. **Ignorar advertencias del propio agente.** Muchos asistentes incluyen advertencias del tipo "esto debería estar en una variable de entorno". En la práctica, esas advertencias se ignoran por la prisa.
5. **Nueva superficie de ataque: MCP.** Los servidores MCP (Model Context Protocol) son un vector emergente. GitGuardian encontró 24.008 secretos únicos expuestos en archivos de configuración de MCP en GitHub público solo en 2025. La documentación oficial de muchos MCPs muestra patrones con API keys hardcodedas; los desarrolladores copian, sustituyen el placeholder por su clave real, y commitean.
6. **Credenciales locales en máquinas de desarrollador.** Los agentes IA con acceso profundo al entorno local (editor, terminal, sistema de archivos, almacenes de credenciales) convierten la máquina del

desarrollador en una superficie de ataque. Ataques de prompt injection o de cadena de suministro pueden extraer credenciales locales a través del agente.

El ataque Shai-Hulud es un ejemplo reciente de este vector: paquetes npm comprometidos que capturaban claves SSH, credenciales cloud, y API tokens de máquinas de desarrolladores precisamente donde las herramientas IA concentraban el acceso.

Cómo enmarcar esto en clase sin asustar: el agente IA no es el problema. El agente IA es un amplificador. Si tenías buenas prácticas de gestión de secretos, el agente te ayuda a ir más rápido sin problemas. Si no las tenías, el agente amplifica el riesgo. La solución no es dejar de usar el agente; es aprender las prácticas que permiten usarlo de forma segura.

10. Lo que el PM senior tiene que interiorizar

Cinco ideas clave que quiero que los alumnos se lleven de esta parte de la sesión. Cada una conecta con una pregunta que pueden hacerse ante cualquier situación futura.

Idea 1: "Que funcione" no es "que esté bien". Cuando el agente dice "ya funciona", la pregunta del PM debe ser: ¿funciona porque está bien construido, o porque ha quitado un obstáculo? Si la respuesta es incierta, revisa antes de hacer commit.

Idea 2: Los repositorios privados no son un refugio. El 35% de repos privados contienen secretos en texto plano. Trata cualquier credencial expuesta en cualquier repositorio como comprometida desde el momento en que se commitea.

Idea 3: El tiempo de reacción es medido en minutos, no en días. Si una credencial se commitea a un repositorio público, asume que está comprometida en cinco minutos. No intentes reaccionar; previene.

Idea 4: Las claves no son todas iguales. Una clave con permisos mínimos expuesta es un incidente. Una clave con AdministratorAccess expuesta es una catástrofe. Aplica el principio de mínimo privilegio siempre: cada clave solo debe tener los permisos que necesita.

Idea 5: Los costes reales trascienden la factura directa. Una fuga de credenciales puede costar 82.000 dólares directos, pero también 4,67 millones de media en costes totales de brecha, 246 días de contención, reputación, posibles multas del RGPD, y la confianza de tus usuarios. El PM debe sumar todas las capas al evaluar el riesgo.

11. Fuentes

Ordenadas por peso y relevancia para la sesión.

Informes anuales con datos agregados:

- GitGuardian, State of Secrets Sprawl 2026: <https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026-pr/>
- GitGuardian, State of Secrets Sprawl 2025: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2025>
- IBM, Cost of a Data Breach Report 2025: <https://www.ibm.com/reports/data-breach>

- Help Net Security, análisis del informe 2026: <https://www.helpnetsecurity.com/2026/04/14/gitguardian-ai-agents-credentials-leak/>
- TFiR, entrevista con Dwayne McDaniel de GitGuardian: <https://tfir.io/ai-code-secret-sprawl-gitguardian/>

Casos específicos utilizables en clase:

- Startup mexicana \$82K Gemini (The Register): https://www.theregister.com/2026/03/03/gemini_api_key_82314_dollar_charge/
- Startup mexicana \$82K Gemini (TechSpot): <https://www.techspot.com/news/111529-stolen-gemini-api-key-turned-180-bill-82000.html>
- Investigación Truffle Security sobre Gemini: <https://trufflesecurity.com/blog/google-api-keys-werent-secrets-but-then-gemini-changed-the-rules>
- Estudiante \$55K Gemini (CSO Online): <https://www.csoonline.com/article/4138749/silent-google-api-key-change-exposed-gemini-ai-data.html>
- Estudiante \$55K Gemini (36Kr, reporte detallado): <https://eu.36kr.com/en/p/3486014581496960>
- AWS \$89K minería (Medium): <https://medium.com/lets-code-future/we-got-a-89k-aws-bill-overnight-heres-what-went-wrong-b5dfd51c9a7e>
- Campaña EleKtra-Leak (The Register): https://www.theregister.com/2023/10/30/cryptojackers_steal_aws_credentials_github/
- 8.000+ claves ChatGPT expuestas (Vicarius): <https://www.vicarius.io/articles/8-000-chatgpt-api-keys-exposed-across-github-production-sites>

Contexto y profundización:

- Snyk, State of Secrets 2026: <https://snyk.io/articles/state-of-secrets/>
- Análisis del factor IA (Help Net Security): <https://www.helpnetsecurity.com/2026/04/14/gitguardian-ai-agents-credentials-leak/>
- IBM Cost of Data Breach, análisis de credenciales (SpyCloud): <https://spycloud.com/blog/6-takeaways-from-ibm-data-breach-report-2025/>

Fin del documento de Familia 1.

Próximos pasos sugeridos:

- Validar con Jaime que la profundidad y el tono encajan.
- Producir la versión ligera para alumnos (guía de referencia resumida).
- Pasar a la Familia 2 (Bases de datos y permisos) con el mismo nivel de profundidad.